RETO DE 25 DÍAS · DÍA 1

SPEC-DRIVEN DEVELOPMENT

Guía práctica para instalar y usar Spec Kit

Antes de escribir una sola línea de código en este reto, vamos a aprender a escribir bien lo que queremos construir. Esta guía te lleva de un prompt suelto a tu primera especificación formal, y de ahí a instalar y usar Spec Kit, la herramienta open source de GitHub para Spec-Driven Development.

CONCEPTO

¿QUÉ ES SPEC-DRIVEN DEVELOPMENT?

Spec-Driven Development (SDD) invierte el orden con el que normalmente se trabaja con IA. En vez de escribirle un prompt suelto a un asistente y esperar que "adivine" lo que quieres, primero escribes una especificación: un documento que define **qué** quieres construir, antes de decidir **cómo** se va a construir.

> *"Intent-driven development, donde las especificaciones definen el qué antes del cómo." — filosofía central de Spec Kit*

La diferencia no es cosmética. Un prompt suelto vive y muere en esa conversación. Una spec es un artefacto que puedes revisar, versionar, discutir con otras personas y usar como fuente de verdad durante todo el proyecto — la IA deja de improvisar y empieza a ejecutar sobre una base clara.

Por qué importa para este reto

Ya viste en el Día 1 que un prompt sin estructura (sin rol, sin contexto, sin formato de salida, sin restricciones) produce resultados mediocres. SDD toma exactamente esa misma lógica de estructura deliberada y la escala: en vez de aplicarla a una sola pregunta, la aplicas a un proyecto completo.

— Reduces la ambigüedad antes de que la IA escriba código.

— Tienes un documento de referencia al que volver cuando el proyecto crece.

— Puedes iterar sobre la spec en vez de reescribir código a ciegas.

HERRAMIENTA

¿QUÉ ES SPEC KIT?

Spec Kit es el toolkit open source de GitHub para practicar Spec-Driven Development con agentes de IA para programar. Es software libre (licencia MIT) y funciona como una capa de comandos y plantillas que se instala sobre tu proyecto y tu agente de IA favorito.

Repositorio oficial

github.com/github/spec-kit

Requisitos antes de instalar

— Linux, macOS o Windows

— Python 3.11 o superior

— Git instalado

— `uv` (recomendado) o `pipx` para instalar el CLI

— Un agente de IA compatible (Spec Kit soporta más de 30: Claude Code, Copilot, Cursor, y otros)

Spec Kit no reemplaza a tu agente de IA: le da una estructura de trabajo. Vive como una carpeta `.specify/` dentro de tu proyecto, con las plantillas que tu agente va a rellenar en cada fase.

PRÁCTICA REAL

LO QUE DE VERDAD DEBE LLEVAR UNA SPEC

GitHub analizó más de 2.500 archivos de especificación reales usados por equipos que ya trabajan con agentes de IA. El patrón se repite: las specs que funcionan cubren siempre seis áreas. Las que fallan, se quedan en generalidades.

1. Comandos

Ejecutables y con flags reales: `npm test`, `pytest -v`. No "correr los tests", sino el comando exacto.

2. Testing

Framework usado, dónde viven los archivos de test, qué cobertura se espera.

3. Estructura del proyecto

Dónde vive el código fuente, los tests y la documentación.

4. Estilo de código

Un fragmento real de código con tu estilo vale más que tres párrafos describiéndolo.

5. Flujo de git

Nombres de rama, formato de commits, qué requiere un PR para mergearse.

6. Límites

Qué zonas del proyecto el agente no debe tocar bajo ninguna circunstancia.

El sistema de límites en tres niveles

En vez de una lista plana de reglas, organiza los límites por nivel de riesgo:

— ✅ **Siempre** — no necesita aprobación. Ej: "corre los tests antes de cada commit".

— ⚠️ **Pregunta primero** — cambios de alto impacto. Ej: "pregunta antes de modificar el esquema de la base de datos".

— 🚫 **Nunca** — límites absolutos. Ej: "nunca subas secretos ni API keys" — la regla más citada como útil en specs reales.

El error más común

La mayoría de specs fallan por ser demasiado vagas. "Construye algo genial" no es una spec. Especifica inputs, outputs y restricciones exactas — y no le metas todo el contexto de golpe en un solo prompt: el rendimiento de la IA baja cuando tiene que satisfacer demasiados requisitos a la vez.

PRÁCTICA REAL

CÓMO ESCRIBIR UNA CONSTITUTION QUE SIRVA

El error número uno al escribir `constitution.md` es llenarlo de frases bonitas que no cambian nada. "El código debe ser legible" o "los tests deben cubrir lo crítico" suenan bien, pero el agente no tiene forma de actuar sobre eso: si una regla es compatible con cualquier implementación posible, no es una regla.

> *El test de validación: borra cada principio y pregúntate — ¿algún plan se vería distinto sin esta regla? Si la respuesta es no, era decoración.*

Principios concretos que sí funcionan

— "Un nuevo proceso, servicio o cola necesita justificación escrita ligada a: sobrecarga de working-set, cómputo genuinamente independiente, latencia geográfica, o independencia organizacional."

— "Un módulo debe ser lo bastante pequeño para que una persona lo borre y reescriba en un día. Sin abstracciones especulativas."

— "Los campos de alta cardinalidad (user id, request id, tenant id) son obligatorios en los logs, no opcionales."

— "Un PR no está terminado hasta que el cambio está en manos de usuarios, es observable y reversible."

Qué dejar fuera de la constitution

Nombres de herramientas y versiones ("usa Postgres 16"), preferencias de framework, reglas de estilo de línea — eso va en la config del linter, no aquí. Si una regla no cambia al menos una decisión o un diff, es decoración.

PASO A PASO

INSTALACIÓN

Instala el CLI `specify-cli` apuntando a una versión concreta del repositorio (reemplaza `vX.Y.Z` por la última versión publicada):

```
# instalación desde el repo, versión fija
uv tool install specify-cli --from git+https://github.com/github/spec-
kit.git@vX.Y.Z

# alternativa: instalar desde PyPI
uv tool install specify-cli
```

Con el CLI instalado, crea tu proyecto de prueba para este ejercicio:

```
specify init mi-proyecto --integration copilot
cd mi-proyecto
```

Cambia `copilot` por el agente que uses. Para ver todas las integraciones disponibles en tu versión:

```
specify integration list
```

Comandos útiles para mantener el CLI al día:

```
specify self check              # revisa si hay actualizaciones
specify self upgrade --dry-run  # previsualiza la actualización
specify self upgrade            # actualiza
```

METODOLOGÍA

EL FLUJO DE TRABAJO

Spec Kit organiza el trabajo en comandos que escribes directamente dentro de tu agente de IA, en este orden:

1. `/speckit.constitution`

   Define los principios del proyecto: qué reglas y estándares no son negociables antes de escribir la primera spec.

2. `/speckit.specify`

   Escribes la especificación: qué quieres construir, para quién y con qué criterios de éxito. Aquí es donde pegas la spec que redactaste con la estructura rol / contexto / tarea / formato / restricciones.

3. `/speckit.plan`

   El agente propone la arquitectura técnica: stack, estructura de archivos, decisiones de diseño.

4. `/speckit.tasks`

   El plan se desglosa en tareas concretas y ejecutables.

5. `/speckit.implement`

   El agente ejecuta las tareas y escribe el código.

6. `/speckit.converge`

   Se valida el resultado contra la spec, el plan y las tareas originales, para confirmar que lo construido es lo que se pidió.

7. **Iteración**

   Repites implementar → converger hasta que el proyecto cumple con la spec.

COMPLEMENTOS

COMANDOS OPCIONALES

Además del flujo principal, hay comandos para reforzar la calidad en cualquier punto del proceso:

`/speckit.clarify` OPCIONAL

Detecta y resuelve zonas ambiguas de la spec antes de pasar a plan.

`/speckit.analyze` OPCIONAL

Valida que spec, plan y tareas sean consistentes entre sí.

`/speckit.checklist` OPCIONAL

Genera un checklist de calidad para revisar antes de dar el proyecto por terminado.

EJEMPLO APLICADO AL DÍA 1

Con tu spec ya reescrita (rol, contexto, tarea, formato, restricciones), el mínimo para probar Spec Kit hoy es:

```
specify init reto-dia1 --integration claude
cd reto-dia1

# dentro de tu agente:
/speckit.constitution   # define 2-3 principios simples del proyecto
/speckit.specify        # pega aquí tu spec reescrita del Día 1
```

No hace falta llegar hasta `/speckit.implement` hoy. El objetivo del día es que veas tu spec estructurada convertirse en la entrada real de una herramienta de SDD.

RESUMEN

CHECKLIST ANTES DE COMENTAR TU ENTREGABLE

- [ ] Tengo el prompt original que usé recientemente para pedir código.
- [ ] Identifiqué qué le faltaba: ¿rol? ¿contexto? ¿formato de salida? ¿restricciones?
- [ ] Reescribí ese prompt como una spec estructurada con los cinco elementos.
- [ ] Mi spec cubre las seis áreas reales: comandos, testing, estructura, estilo, git, límites.
- [ ] Mis límites están organizados en Siempre / Pregunta primero / Nunca.
- [ ] Instalé Spec Kit y corrí `specify init .`
- [ ] Escribí mi `constitution.md` aplicando el test: borré cada principio y comprobé que sí cambiaba una decisión.
- [ ] Ejecuté `/speckit.constitution` y `/speckit.specify` con mi spec.
- [ ] Tomé una captura del resultado para comentar en el post del Día 1.

Repositorio oficial: github.com/github/spec-kit